

Informe Técnico: RadixSort

RadixSort es un algoritmo de ordenamiento **no comparativo** que organiza los elementos procesando sus dígitos individuales. Generalmente trabaja de **menor a mayor significancia (LSD: Least Significant Digit)**.

Su funcionamiento se basa en distribuir los elementos en **estructuras auxiliares (buckets)** según el valor de cada dígito, para luego reconstruir la secuencia ordenada. Este proceso se repite por cada posición del número.

Conceptualmente, puede interpretarse como una serie de ordenamientos parciales que progresivamente construyen el orden final.



Implementaciones y Comparativa

♦ RadixSort con Array

Características:

- Acceso aleatorio en tiempo constante ($O(1)$)
- Memoria contigua
- Uso típico de estructuras auxiliares (conteo o buckets)

Ventajas:

- Buen rendimiento general
- Excelente localidad de memoria
- Implementación eficiente

Desventajas:

- Requiere estructuras auxiliares adicionales
- Posibles costos de copia entre arrays

Conclusión:

Es una implementación **eficiente y estándar**, aunque puede verse afectada por movimientos de datos entre estructuras auxiliares.

♦ RadixSort con ArrayList

Características:

- Basado internamente en arrays
- Acceso aleatorio ($O(1)$)

Ventajas:

- Flexibilidad en tamaño
- Código más abstracto y manejable

Desventajas:

- Ligera sobrecarga por gestión dinámica
- Redimensionamientos ocasionales

Conclusión:

Presenta un rendimiento **muy cercano al array**, con una pequeña penalización por abstracción.

RadixSort con LinkedList

Características:

- Inserciones y eliminaciones en $O(1)$
- Memoria no contigua
- Acceso secuencial

Ventajas:

- Inserciones extremadamente eficientes (clave en RadixSort)
- Ideal para implementación basada en buckets dinámicos

Desventajas:

- Baja localidad de memoria
- Mayor uso de memoria por nodos

Conclusión:

A diferencia de HeapSort, [LinkedList puede ser altamente eficiente en RadixSort](#), dependiendo de la implementación.



Complejidad Algorítmica

La complejidad de RadixSort es:

[
 $O(n \cdot k)$
]

Donde:

- (n): número de elementos
- (k): cantidad de dígitos del número mayor

Resultado:

- Mejor caso: ($O(n \cdot k)$)
- Caso promedio: ($O(n \cdot k)$)
- Peor caso: ($O(n \cdot k)$)

Conclusión:

RadixSort presenta un comportamiento [lineal respecto a n](#), si (k) es constante.



Resultados de Laboratorio

Datos Aleatorios

- LinkedList presenta el mejor rendimiento en todos los tamaños

- Array y ArrayList tienen tiempos similares

Ejemplo destacado:

Size	Array	ArrayList	LinkedList
100,000	0.017283	0.015910	0.009412
1,000,000	0.178901	0.171203	0.088089
2,000,000	0.340396	0.329797	0.177190



Datos Invertidos

- Comportamiento prácticamente idéntico al caso aleatorio
- LinkedList sigue siendo superior

Ejemplo:

Size	Array	ArrayList	LinkedList
1,000,000	0.244171	0.243705	0.087714
2,000,000	0.495269	0.493742	0.177155



Datos Ordenados

- No hay mejora significativa
- El algoritmo no depende del orden inicial

Ejemplo:

Size	Array	ArrayList	LinkedList
1,000,000	0.220604	0.217243	0.101364

Size	Array	ArrayList	LinkedList
2,000,000	0.491179	0.499475	0.197653



Análisis de Resultados

Observación 1: LinkedList es más rápido

A diferencia de HeapSort, este resultado **sí es coherente con RadixSort**, porque:

- ✓ El algoritmo realiza muchas inserciones en buckets
- ✓ LinkedList permite inserciones en $O(1)$
- ✓ No requiere desplazamiento de elementos

👉 Resultado: ventaja clara para LinkedList



Observación 2: Array vs ArrayList

- Comportamiento muy similar
- Diferencias leves al crecer (n)

Explicación:

- Overhead de ArrayList
- Posibles redimensionamientos



Observación 3: Independencia del orden

- Aleatorio, ordenado e invertido → resultados similares

👉 Confirma que RadixSort:

- ✓ No depende del orden inicial
- ✓ Mantiene comportamiento uniforme



Observación 4: Escalabilidad

Los tiempos crecen de forma aproximadamente:

[
O(n)
]

- ✓ Totalmente consistente con RadixSort



Discusión Técnica

El rendimiento de RadixSort depende fuertemente de **cómo se implementan los buckets**:

- **Array**: requiere copiar datos entre estructuras
- **ArrayList**: similar al array, con leve overhead
- **LinkedList**: permite construir buckets dinámicos sin copias costosas

Esto explica por qué:

// LinkedList supera a Array en esta implementación



Conclusión General

RadixSort es un algoritmo:

- **Eficiente:** cercano a lineal
- **Consistente:** independiente del input
- **Flexible:** adaptable a distintas estructuras

Sin embargo:

- Depende fuertemente de la implementación
- Puede consumir memoria adicional
- No siempre supera a algoritmos comparativos en práctica



Síntesis Final

- 1 RadixSort (según resultados experimentales):
- 2
- 3 `LinkedList > Array ≈ ArrayList`



Conclusión Final de la Práctica

- La estructura de datos influye directamente en el rendimiento
- En RadixSort, las operaciones de inserción son críticas
- LinkedList puede ser la mejor opción si el algoritmo está orientado a buckets dinámicos